# Project Executable Files

| | |
|---|---|
| **Date** | 15 March 2026 |
| **Team ID** | |
| **Project Name** | Credit Card Approval Prediction |
| **Maximum Marks** | 3 Marks |

## Project Executable Files

### Step 1: Submission Checklist

| S.No | Item to Submit | Submitted(Yes/ No / NA) |
|---|---|---|
| 1 | Complete source code (all files and folders) | Yes |
| 2 | README / Setup Guide (instructions to run the project) | Yes |
| 3 | requirements.txt / package.json / dependency file | Yes |
| 4 | Database schema / seed files (if applicable) | Yes |
| 5 | Environment configuration file (.env.example or similar) | NA |
| 6 | Deployed application URL (if hosted) | **No** (Local Deployment) |
| 7 | APK / Executable binary (if applicable for mobile/desktop apps) | NA |
| 8 | Dockerfile / Containerization config (if applicable) | NA |
| 9 | Test files and test results | Yes |
| 10 | Demo video or walkthrough (if required) | Yes |

**Step 2: File / Folder Structure**

```
Credit_Card_Project/
│
├── 1. Brainstorming & Ideation/        - Initial project ideas and problem identification.
│   ├── Brainstorming.pdf              - Brainstorming discussion.
│   ├── Literature Survey.pdf          - Existing research review.
│   ├── Problem Statement.pdf          - Defines the project problem.
│   └── Empathy Map.pdf                - User needs and pain points.
│
├── 2. Requirement Analysis/            - Project requirements.
│   ├── Customer Journey.pdf           - User interaction flow.
│   ├── Data Flow Diagram.pdf          - System data flow.
│   ├── Functional Requirements.pdf    - System functionalities.
│   └── Technology Stack.pdf           - Technologies used.
│
├── 3. Project Design Phase/            - System design documents.
│   ├── Solution Architecture.pdf      - Overall system architecture.
│   ├── Problem Solution Fit.pdf       - Problem and solution mapping.
│   ├── Proposed Solution.pdf          - Proposed system details.
│   └── Sprint Planning.pdf            - Sprint schedule.
│
├── 4. Project Planning Phase/          - Project planning documents.
│   ├── Milestone Planning.pdf         - Project milestones.
│   ├── Project Planning.pdf           - Development plan.
│   └── Sprint Schedule.pdf            - Sprint timeline.
│
├── 5. Project Development Phase/       - Source code implementation.
│   └── Source Code/
│       ├── static/                    - Static resources.
│       │   └── style.css              - Application styling.
│       │
│       ├── templates/                 - HTML web pages.
│       │   ├── index.html             - Home page.
│       │   ├── result.html            - Prediction result page.
│       │   ├── history.html           - Prediction history page.
│       │   ├── dashboard.html         - Analytics dashboard.
│       │   └── login.html             - Login page.
│       │
│       ├── app.py                     - Flask application.
│       ├── main.py                    - Model training script.
│       ├── model.pkl                  - Trained ML model.
│       ├── credit_card.db             - SQLite database.
│       ├── application_record.csv     - Applicant dataset.
│       ├── credit_record.csv          - Credit history dataset.
│       ├── requirements.txt           - Required Python packages.
│       └── README.md                  - Project instructions.
│
```

```
├── 6. Project Testing/             - Testing documents.
│   ├── Test Cases.pdf              - Test case details.
│   ├── Application Building.pdf    - Build process.
│   └── Output Screenshots.pdf      - Application screenshots.
│
├── 7. Project Documentation/       - Project documentation.
│   ├── Final Report.pdf            - Final project report.
│   └── Project Presentation.pptx   - Project presentation.
│
├── 8. Project Demonstration/       - Demonstration files.
│   ├── Demo Video.mp4              - Project demo video.
│   └── Presentation.pptx           - Demonstration slides.
│
├── README.md                       - Project overview.
├── requirements.txt                - Dependency list.
└── .gitignore                      - Git ignore configuration.
```

**Step 3: Deployment / Access Details**

| Field | Details |
|---|---|
| **Hosted / Deployed URL** | Not Deployed (Runs on Local Flask Server) |
| **Login Credentials (Demo)** | Not Applicable |
| **Platform / Hosting Provider** | Local Flask Development Server (Python + Flask + SQLite) |
| **Repository Link** | https://github.com/Shaik-Johnsaida/Credit_Card_Approval_Prediction |
| **Demo Video Link** | https://drive.google.com/drive/folders/1egLKtGw_mnt1K2rvoBMAeKbtIxdjuu7Z?usp=sharing |

### Step 4: Run Instructions

**Pre-requisites:** Python 3.10 or above, Visual Studio Code, Flask, NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, SQLite3, and a web browser (Google Chrome or Microsoft Edge).

**Step 1:** Open the **Credit_Card_Project** folder in Visual Studio Code
.

**Step 2:** Create and activate the virtual environment using the commands: "python -m venv .venv" and ".venv\Scripts\activate" or ".\.venv\Scripts\Activate.ps1".

**Step 3:** Install all the required Python libraries using the command: "pip install -r requirements.txt".

**Step 4:** Start the Flask application using the command: "python app.py".

**Step 5:** Open your web browser and navigate to http://127.0.0.1:5000/.

**Step 6:** Enter the applicant details in the form, click **Predict Credit Approval**, view the prediction result along with the confidence score, access the **Prediction History** to view saved records, and open the **Dashboard** to view prediction statistics and analytics.

### Step 5: Known Issues / Limitations

| S.No | Known Issue / Limitation | Workaround / Status |
|---|---|---|
| 1 | Application currently runs only on the local Flask development server. | Deploy the application to a cloud platform such as Render, Railway, or Azure. |
| 2 | SQLite database is suitable only for small to medium-sized applications. | Upgrade to MySQL or PostgreSQL for large-scale deployment. |
| 3 | The application requires a local Python environment to execute. | Package the application using Docker or deploy it as a web service for easier access. |
| 4 | Basic login authentication is implemented. | Add password encryption, email verification, and role-based access control. |